

ROBOT GRID NAVIGATION & MULTI-ROBOT WAREHOUSE PLANNER

Project Report • Artificial Intelligence / Search and Planning

Project summary. This project models warehouse navigation as a grid-based search problem and extends it with multi-robot task assignment, schedule/conflict validation, action planning, a simulated human-proximity safety rule, and movement-uncertainty analysis. The implementation is a software simulation, not a connection to physical robots.

1. Problem Formulation

A warehouse is represented as a grid containing traversable cells and blocked cells. A robot must find a valid route from a start cell to a target cell while avoiding obstacles. The extension considers multiple robots and warehouse tasks such as travelling to a pickup location. The planner must assign tasks to eligible robots and prevent unsafe schedule conflicts.

Element	Definition
State	The robot's current grid coordinate (row, column); for multi-robot planning, the state also includes task/robot assignment and schedule time where relevant.
Initial state	The configured start cell for each robot and the initial set of unassigned tasks.
Actions	Move to a legal adjacent cell (up, down, left, right); higher-level STRIPS-style actions include Move, Pick, Drop and Charge.
Goal test	The robot reaches its target/pickup cell, or all scheduled tasks in the scenario have been completed.
Path cost	Each grid move has unit cost. A route's cost is its number of steps; the multi-robot schedule is evaluated using makespan (the finishing time of the last robot).
Constraints	Avoid blocked cells; assign tasks only to eligible robots; reject shared-cell and edge-swap conflicts; apply a simulated human-proximity margin.

2. Approach

Breadth-First Search (BFS). BFS explores the grid in increasing number of moves. With equal-cost moves, the first route found is a shortest path. It is straightforward to implement and provides a dependable baseline for an unweighted grid.

Constraint Satisfaction Problem (CSP). Task-to-robot assignment uses domains and constraints. AC-3 removes values that cannot satisfy a binary constraint; backtracking is used to find a consistent assignment when needed.

2. Approach (continued)

Multi-robot scheduling. The planner creates schedules for assigned tasks and validates them for shared-cell occupancy and edge swaps (two robots crossing the same edge in opposite directions at the same time). Waiting can be introduced to avoid a conflict. In the recorded run, Robot R2 waits at cell (3, 7) at time 11.

Action planning, safety and uncertainty. A STRIPS-style action catalogue represents Move, Pick, Drop and Charge actions. The safety module demonstrates a human-proximity margin around a configured illustrative position. The uncertainty module assumes a movement success probability of 0.8 and estimates attempts as steps divided by success probability. These are modelling assumptions, not sensor measurements.

Heuristic / evaluation function. BFS does not require a heuristic. For comparisons, path length counts moves; for multi-robot schedules, makespan is the maximum finishing time across robots. The uncertainty analysis compares candidate routes by steps and expected attempts. No classification accuracy or game win-rate is defined for this planning task, so those metrics are not applicable.

3. Complexity and Practical Limits

Component	Time complexity	Space complexity	Practical limit
BFS grid search	$O(V + E)$; for a 4-neighbour grid, $O(RC)$	$O(V)$	Stores visited cells/frontier; large grids increase memory use.
CSP assignment	Worst case exponential, approximately $O(d^n)$ search; AC-3 commonly $O(e d^3)$	$O(n d)$ for domains, excluding search stack	Backtracking can grow quickly as tasks, robots and domains increase.
Schedule validation	$O(T)$ for a pass over T scheduled time steps/positions	$O(T)$ if the schedule is stored	Long schedules and many robots increase validation work.

Practical limits encountered. The tested environment is a small, fixed simulation. The human location is illustrative; there is no live camera or proximity sensor. The 0.8 movement-success probability is assumed rather than measured. The single-robot baseline is hypothetical and does not apply the same zone restrictions as the multi-robot case. The Charge action is represented in the action catalogue but is not integrated into the active scheduler.

4. Results

Table 1. Recorded run results

Metric	Observed result	Meaning
BFS path length	8 steps	Number of moves in the recorded route.
BFS nodes expanded	18	Search effort in the recorded run.
BFS runtime	0.000468 s	Recorded runtime; depends on machine and run conditions.
Automated tests	3 passed	Path exists, no path, and obstacle avoidance tests.
Robot R1 finish time	27	Recorded multi-robot schedule.
Robot R2 finish time	13	Includes a wait to avoid conflict.
Multi-robot makespan	27 time units	Maximum robot finish time.
Single-robot baseline	37 time units	Hypothetical comparison; different zone constraints.
Makespan difference	10 units (about 27.0%)	Reduction relative to the 37-unit illustrative baseline.

4. Results (continued)

Table 2. Representative console output excerpts

The following excerpts reproduce key lines from the recorded terminal runs; they are text excerpts rather than captured screenshots.

```
BFS baseline
Path length: 8
Nodes expanded: 18
Runtime: 0.000468 seconds

Automated tests
3 passed in 0.05s

MULTI-ROBOT WAREHOUSE PLANNER
CSP assignment: T1 -> R1, T2 -> R2, T3 -> R1
R1 finish time: 27
R2 finish time: 13
Makespan: 27
CONFLICT VALIDATION:
No shared-cell or edge-swap conflicts found.

UNCERTAINTY ANALYSIS
Movement success probability: 80%
Alternative 1: 6 steps; expected attempts 7.50
Alternatives 2/3: 8 steps; expected attempts 10.00
Hypothetical single-robot time: 37
Multi-robot makespan: 27
```

Metric interpretation

The recorded multi-robot makespan is 27 time units compared with 37 for the illustrative single-robot baseline, a difference of 10 units (about 27.0%). This is not a controlled benchmark because the baseline ignores the multi-robot zone restrictions. The conflict validator reported no shared-cell or edge-swap conflicts for the tested schedule. Accuracy and win-rate are not applicable because the program performs search and planning rather than prediction or competitive gameplay.

5. Reflection

The most challenging part was combining independent AI ideas—shortest-path search, constraint-based task assignment, time-based scheduling and conflict checks—while keeping their assumptions consistent. Debugging module integration and checking that the schedule avoided collisions were important parts of the extension. Unit tests helped confirm the core BFS behaviour, while printed schedule output made the multi-robot result easier to inspect.

Possible improvements: add a graphical animation of robot routes and waits; benchmark several grid sizes and obstacle densities; measure runtime across repeated runs; add more CSP and schedule tests; integrate charging into scheduling; and replace illustrative human-position and movement-success assumptions with real sensor data if hardware becomes available.

Honesty note on AI/source help: AI assistance was used to help organize and edit the report text; project behaviour and numerical results are based on the recorded program outputs and tests. Any externally sourced code or material should be cited separately if used.

Reproducibility note. The reported numbers are from the recorded runs available during report preparation. Runtime can vary between machines and executions. Keep the source code, test output, and final README in the repository so the results can be checked.